

Ashwath Raj M | AP25110010712 | Core CSE A

GITHUB LINK: <https://github.com/Ashwath-Raj/CSE-204-DAA-assignments->

1. Implement a Hash Table Using Linear Probing

Problem

Implement a hash table using Linear Probing for handling collisions.

Method

The hash table uses the hash function $\text{key} \% \text{size}$. If the calculated position is already occupied, the next available position is searched sequentially using Linear Probing.

Output:

```
Enter Hash Table Size: 5
Enter Number of Elements: 3
Enter Element 1: 4
Enter Element 2: 8
Enter Element 3: 10
Hash Table:
0 : 10
1 : -1
2 : -1
3 : 8
4 : 4
```

Code:

```
/*
Program Name: Hash Table Using Linear Probing
Program Description: Takes the inputs {keys, table size} and implements a
hash table using Linear Probing for collision handling
Program Creation Time: Aug 27th 3:10
Program Creator: Ashwath Raj | AP25110010712
*/
```

```
#include<iostream>
using namespace std;
```

```
class HashTable{
int *table;
int size;
```

```

public:
HashTable(int n){
size=n;
table=new int[size];

for(int i=0;i<size;i++){
table[i]=-1;
}

void insert(int key){
int idx=key%size;

while(table[idx]!=-1)
idx=(idx+1)%size;

table[idx]=key;
}

void display(){
cout<<"Hash Table:"<<endl;

for(int i=0;i<size;i++){
cout<<i<<" : "<<table[i]<<endl;
}
};

int main(void){
int size,n,key;

cout<<"Enter Hash Table Size: ";
cin>>size;

HashTable ht(size);

cout<<"Enter Number of Elements: ";
cin>>n;

for(int i=0;i<n;i++){
cout<<"Enter Element "<<i+1<<": ";
cin>>key;
ht.insert(key);
}

ht.display();

return 0;
}

```

Complexity

Average Time Complexity: $O(1)$

Worst Case Time Complexity: $O(n)$

Space Complexity: $O(n)$

2. Implement a Hash Table Using Quadratic Probing

Problem

Implement a hash table using Quadratic Probing for handling collisions.

Method

The hash table uses the hash function $\text{key} \% \text{size}$. If the calculated position is already occupied, new positions are searched using quadratic increments.

Output:

```
Enter Hash Table Size: 5
Enter Number of Elements: 3
Enter Element 1: 2
Enter Element 2: 22
Enter Element 3: 122
Hash Table:
0 : -1
1 : 122
2 : 2
3 : 22
4 : -1
```

Code:

```
/*
Program Name: Hash Table Using Quadratic Probing
Program Description: Takes the inputs {keys, table size} and implements a
hash table using Quadratic Probing for collision handling
Program Creation Time: Aug 27th 3:50
Program Creator: Ashwath Raj | AP25110010712
*/
```

```
#include<iostream>
using namespace std;
```

```
class HashTable{
int *table;
```

```
int size;
```

```
public:
```

```
HashTable(int n){
```

```
size=n;
```

```
table=new int[size];
```

```
for(int i=0;i<size;i++)
```

```
table[i]=-1;
```

```
}
```

```
void insert(int key){
```

```
int idx=key%size;
```

```
int i=0;
```

```
while(table[(idx+i*i)%size]!=-1)
```

```
i++;
```

```
table[(idx+i*i)%size]=key;
```

```
}
```

```
void display(){
```

```
cout<<"Hash Table:"<<endl;
```

```
for(int i=0;i<size;i++)
```

```
cout<<i<<" : "<<table[i]<<endl;
```

```
}
```

```
};
```

```
int main(void){
```

```
int size,n,key;
```

```
cout<<"Enter Hash Table Size: ";
```

```
cin>>size;
```

```
HashTable ht(size);
```

```
cout<<"Enter Number of Elements: ";
```

```
cin>>n;
```

```
for(int i=0;i<n;i++){
```

```
cout<<"Enter Element "<<i+1<<": ";
```

```
cin>>key;
```

```
ht.insert(key);
```

```
}
```

```
ht.display();
```

```
return 0;
```

```
}
```

Complexity

Average Time Complexity: $O(1)$

Worst Case Time Complexity: $O(n)$

Space Complexity: $O(n)$

3. Find Two Elements Whose Sum is Equal to Target {T}

Problem

Given an array and a target value T, determine whether there are two elements whose sum is equal to T.

Method

Hashing is used to store the elements already visited. For every element, its required complement $T - arr[i]$ is searched in the hash table. If the complement is found, two elements with the required sum exist.

Output:

```
Enter Size of Array: 5
Enter Element 1: 2
Enter Element 2: 7
Enter Element 3: 5
Enter Element 4: 9
Enter Element 5: 11
Enter Target Value: 9
Two elements with the given sum exist!!
```

Code:

```
/*
Program Name: Two Sum Using Hashing
Program Description: Takes the inputs {array, array size, target} and
determines whether two elements have sum equal to target
Program Creation Time: Aug 27th 4:00
Program Creator: Ashwath Raj | AP25110010712
*/
```

```
#include<iostream>
#include<unordered_set>
using namespace std;
```

```
bool twoSum(int arr[],int n,int target){
```

```

unordered_set<int> hash;

for(int i=0;i<n;i++){
    int req=target-arr[i];

    if(hash.find(req)!=hash.end())
        return true;

    hash.insert(arr[i]);
}

return false;
}

int main(void){
    int n,target;

    cout<<"Enter Size of Array: ";
    cin>>n;

    int arr[n];

    for(int i=0;i<n;i++){
        cout<<"Enter Element "<<i+1<<" ";
        cin>>arr[i];
    }

    cout<<"Enter Target Value: ";
    cin>>target;

    if(twoSum(arr,n,target))
        cout<<"Two elements with the given sum exist!!"<<endl;
    else
        cout<<"No such pair exists!!"<<endl;

    return 0;
}

```

Complexity

Average Time Complexity: $O(n)$

Worst Case Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

4. Find Frequency of Every Distinct Element Using Hashing

Problem

Given an integer array, use hashing to find the frequency of every distinct element.

Method

A hash table is used to store each distinct element as a key and its frequency as the corresponding value. The frequency is incremented whenever the element occurs in the array.

Output:

```
Enter Size of Array: 5
Enter Element 1: 3
Enter Element 2: 2
Enter Element 3: 1
Enter Element 4: 2
Enter Element 5: 3
Element Frequency:
1 : 1
2 : 2
3 : 2
```

Code:

```
/*
Program Name: Frequency of Elements Using Hashing
Program Description: Takes an integer array and uses hashing to find the
frequency of every distinct element
Program Creation Time: Aug 27th 4:20
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
#include<unordered_map>
using namespace std;

void findFrequency(int arr[],int n){
    unordered_map<int,int> freq;

    for(int i=0;i<n;i++){
        freq[arr[i]]++;
    }

    cout<<"Element Frequency:"<<endl;

    for(auto x:freq)
        cout<<x.first<<" : "<<x.second<<endl;
```

```
}

int main(void){
int n;

cout<<"Enter Size of Array: ";
cin>>n;

int arr[n];

for(int i=0;i<n;i++){
cout<<"Enter Element "<<i+1<<" ";
cin>>arr[i];
}

findFrequency(arr,n);

return 0;
}
```

Complexity

Average Time Complexity: $O(n)$

Worst Case Time Complexity: $O(n^2)$

Space Complexity: $O(n)$